Q1. Student Performance and Correlation Analysis A college wants to identify which academic factors have the strongest relationships with examination marks.

```
# (a) Calculate the Pearson correlation matrix and create an annotated heatmap.
#  Rank all four features by their absolute correlation with Marks, identify the strongest positive
# and negative relationships, and explain what the coefficients indicate.
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.DataFrame({
    "Study_Hours": [2, 3, 4, 5, 6, 7, 8, 9, 3, 6, 8, 4],
    "Attendance": [55, 60, 65, 70, 75, 80, 85, 95, 62, 78, 90, 68],
    "Assignments": [3, 4, 5, 5, 6, 7, 8, 10, 4, 7, 9, 5],
    "Sleep_Hours": [9, 8, 8, 7, 7, 6, 6, 5, 9, 7, 5, 8],
    "Marks": [40, 45, 50, 58, 65, 72, 78, 95, 43, 70, 90, 52]
})

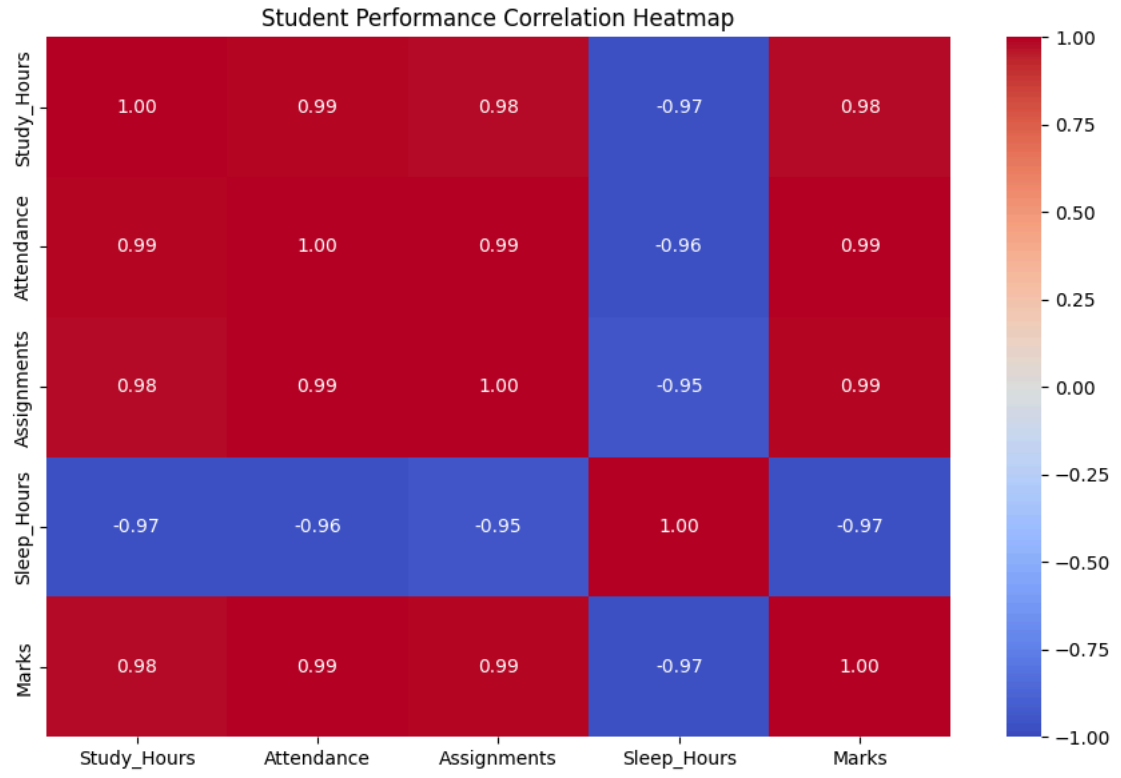
corr_matrix = df.corr()

print(corr_matrix.round(4))

# Create annotated heatmap
plt.figure(figsize=(9, 6))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    vmin=-1,
    vmax=1
)

plt.title("Student Performance Correlation Heatmap")
plt.tight_layout()
plt.show()
```

```
             Study_Hours  Attendance  Assignments  Sleep_Hours   Marks
Study_Hours       1.0000      0.9896       0.9768      -0.9685  0.9815
Attendance        0.9896      1.0000       0.9931      -0.9621  0.9930
Assignments       0.9768      0.9931       1.0000      -0.9525  0.9907
Sleep_Hours      -0.9685     -0.9621      -0.9525       1.0000 -0.9738
Marks             0.9815      0.9930       0.9907      -0.9738  1.0000
```




```
# (b) Remove the student with the highest marks and recalculate the correlation matrix.
#  Compare the feature-to-marks correlations before and after removal, and explain how a single observation can influence t

highest_index = df["Marks"].idxmax()
```

```
df_without_topper = df.drop(index=highest_index)

new_corr_matrix = df_without_topper.corr()

print("Correlation matrix after removing highest-mark student:")
print(new_corr_matrix.round(4))

comparison = pd.DataFrame({
    "Before Removal": df.corr()["Marks"],
    "After Removal": new_corr_matrix["Marks"]
})

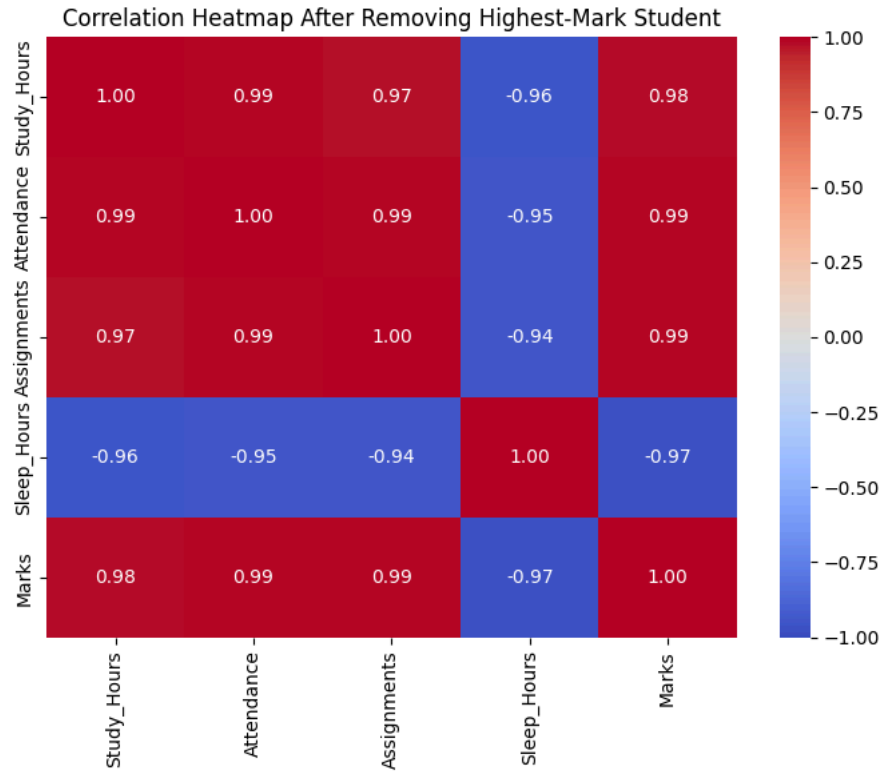
print("\nComparison with Marks:")
print(comparison.drop("Marks").round(4))

plt.figure(figsize=(7, 6))
sns.heatmap(
    new_corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    vmin=-1,
    vmax=1
)

plt.title("Correlation Heatmap After Removing Highest-Mark Student")
plt.tight_layout()
plt.show()
```

```
Correlation matrix after removing highest-mark student:
              Study_Hours  Attendance  Assignments  Sleep_Hours   Marks
Study_Hours        1.0000      0.9877       0.9732      -0.9591  0.9771
Attendance         0.9877      1.0000       0.9909      -0.9528  0.9901
Assignments        0.9732      0.9909       1.0000      -0.9432  0.9871
Sleep_Hours       -0.9591     -0.9528      -0.9432       1.0000 -0.9692
Marks              0.9771      0.9901       0.9871      -0.9692  1.0000

Comparison with Marks:
              Before Removal  After Removal
Study_Hours           0.9815         0.9771
Attendance            0.9930         0.9901
Assignments           0.9907         0.9871
Sleep_Hours          -0.9738        -0.9692
```



Q2. Detecting Redundant Features in a Housing Dataset A property company wants to investigate relationships among housing attributes before building a machine learning model.

```
# (a) Generate a correlation matrix and heatmap. Programmatically identify unique feature pairs whose absolute
#  correlation exceeds 0.90, excluding self-correlations and duplicate pairs. Explain why two highly correlated features ma
df = pd.DataFrame({
    "Area_sqft": [800, 1000, 1200, 1400, 1600, 1800, 2000, 2200],
    "Area_sqm": [74.32, 92.90, 111.48, 130.06, 148.64, 167.22, 185.80, 204.38],
    "Bedrooms": [1, 2, 2, 3, 3, 3, 4, 4],
    "Age_Years": [25, 20, 18, 15, 12, 10, 8, 6],
    "Price_Lakhs": [25, 32, 38, 45, 52, 60, 68, 75]
})
corr_matrix = df.corr()

print("Correlation Matrix:")
print(corr_matrix.round(4))

plt.figure(figsize=(7, 6))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    vmin=-1,
    vmax=1
)
plt.title("Housing Dataset Correlation Heatmap")
plt.tight_layout()
plt.show()

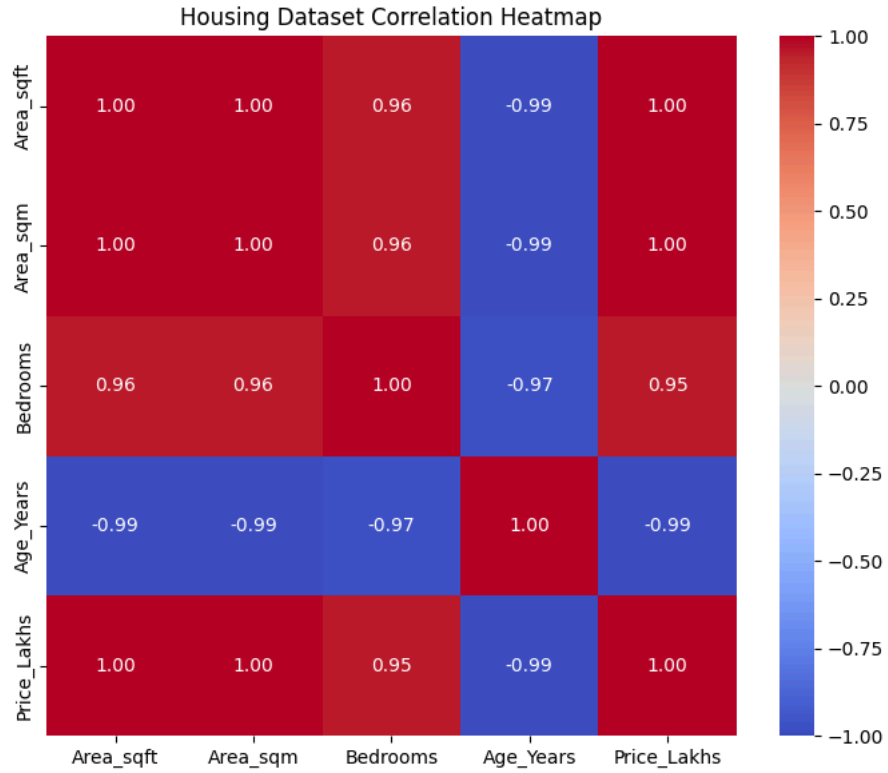
threshold = 0.90
pairs = []

for i in range(len(corr_matrix.columns)):
    for j in range(i + 1, len(corr_matrix.columns)):
        corr_value = corr_matrix.iloc[i, j]

        if abs(corr_value) > threshold:
            pairs.append((
                corr_matrix.columns[i],
                corr_matrix.columns[j],
                corr_value
            ))

print("\nHighly Correlated Feature Pairs:")
for feature1, feature2, value in pairs:
    print(f"{feature1} - {feature2}: {value:.4f}")
```

```
Correlation Matrix:
             Area_sqft  Area_sqm  Bedrooms  Age_Years  Price_Lakhs
Area_sqft       1.0000    1.0000    0.9578    -0.9907       0.9992
Area_sqm        1.0000    1.0000    0.9578    -0.9907       0.9992
Bedrooms        0.9578    0.9578    1.0000    -0.9698       0.9548
Age_Years      -0.9907   -0.9907   -0.9698     1.0000      -0.9863
Price_Lakhs     0.9992    0.9992    0.9548    -0.9863       1.0000
```




```
Highly Correlated Feature Pairs:
Area_sqft - Area_sqm: 1.0000
Area_sqft - Bedrooms: 0.9578
Area_sqft - Age_Years: -0.9907
Area_sqft - Price_Lakhs: 0.9992
Area_sqm - Bedrooms: 0.9578
Area_sqm - Age_Years: -0.9907
Area_sqm - Price_Lakhs: 0.9992
Bedrooms - Age_Years: -0.9698
Bedrooms - Price_Lakhs: 0.9548
Age_Years - Price_Lakhs: -0.9863
```

```
# (b) Create two feature sets, one containing both area measurements and another containing only Area_sqft.
#  Compare the feature relationships and explain why correlation can help identify redundancy but cannot, by itself, prove

features_A = df[
    ["Area_sqft", "Area_sqm", "Bedrooms", "Age_Years"]
]
features_B = df[
    ["Area_sqft", "Bedrooms", "Age_Years"]
]

print("Feature Set A: Both area measurements")
print(features_A.corr().round(4))

print("\nFeature Set B: Only Area_sqft")
print(features_B.corr().round(4))

print("\nCorrelation with Price_Lakhs:")
print(df.corr()["Price_Lakhs"].drop("Price_Lakhs").round(4))
```

```
Feature Set A: Both area measurements
            Area_sqft  Area_sqm  Bedrooms  Age_Years
Area_sqft      1.0000    1.0000    0.9578    -0.9907
Area_sqm       1.0000    1.0000    0.9578    -0.9907
Bedrooms       0.9578    0.9578    1.0000    -0.9698
Age_Years     -0.9907   -0.9907   -0.9698     1.0000

Feature Set B: Only Area_sqft
            Area_sqft  Bedrooms  Age_Years
Area_sqft      1.0000    0.9578    -0.9907
Bedrooms       0.9578    1.0000    -0.9698
Age_Years     -0.9907   -0.9698     1.0000
```

```
Correlation with Price_Lakhs:
Area_sqft    0.9992
Area_sqm     0.9992
Bedrooms     0.9548
Age_Years   -0.9863
Name: Price_Lakhs, dtype: float64
```

Q3. Customer Spending Analysis Using the Tips Dataset Use Seaborn's built-in tips dataset to analyze restaurant bills and tips.

```
# (a) Select total_bill, tip, and size. Calculate their Pearson correlation matrix and display an annotated heatmap.
#  Identify the strongest relationship and support your interpretation with the calculated coefficients
df = sns.load_dataset("tips")

data = df[["total_bill", "tip", "size"]]

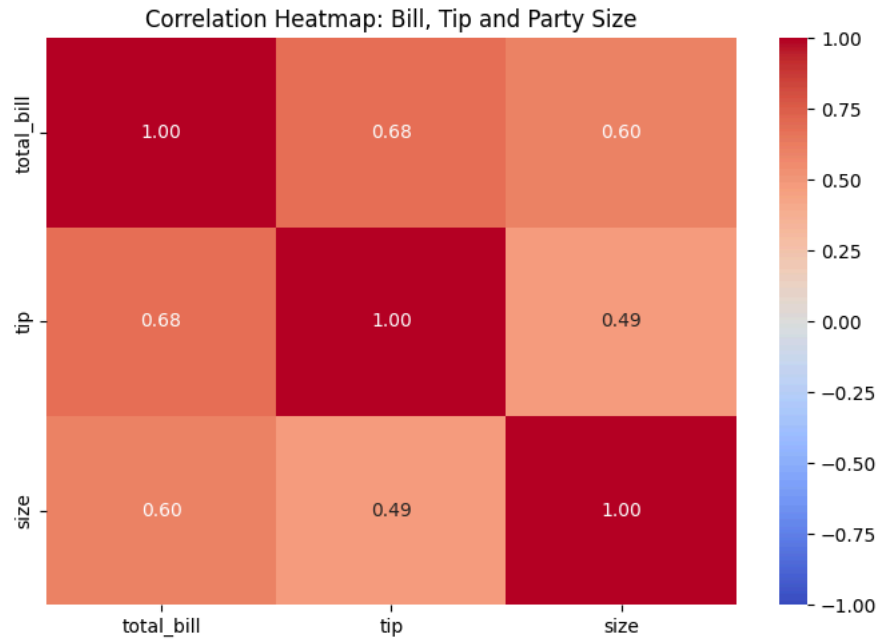

corr_matrix = data.corr()

print("Correlation Matrix:")
print(corr_matrix.round(4))


plt.figure(figsize=(7, 5))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    vmin=-1,
    vmax=1
)

plt.title("Correlation Heatmap: Bill, Tip and Party Size")
plt.tight_layout()
plt.show()
```

```
Correlation Matrix:
            total_bill     tip    size
total_bill      1.0000  0.6757  0.5983
tip             0.6757  1.0000  0.4893
size            0.5983  0.4893  1.0000
```




```
# (b) Create a tip_percentage column using tip / total_bill * 100. Recalculate the correlation matrix and explain how this
#  total_bill and tip. Discuss why mathematical dependencies between variables can complicate the interpretation of correla
df = sns.load_dataset("tips")

data = df[["total_bill", "tip", "size"]]

corr_matrix = data.corr()

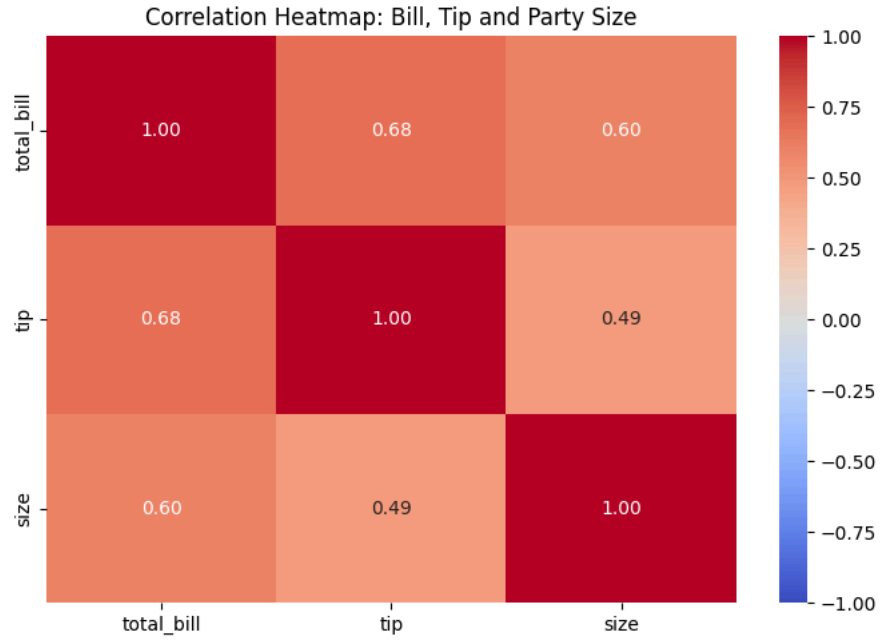
print("Correlation Matrix:")
print(corr_matrix.round(4))

plt.figure(figsize=(7, 5))
```

```
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    vmin=-1,
    vmax=1
)

plt.title("Correlation Heatmap: Bill, Tip and Party Size")
plt.tight_layout()
plt.show()
```

```
Correlation Matrix:
            total_bill     tip    size
total_bill      1.0000  0.6757  0.5983
tip             0.6757  1.0000  0.4893
size            0.5983  0.4893  1.0000
```



Q4. Investigating the Effect of an Outlier A marketing analyst investigates the relationship between advertising expenditure and sales.

```
# (a) Calculate the correlation matrix and plot a heatmap. Add a new observation with advertising expenditure of 100 and
#  sales of 10, then recalculate and visualize the correlations. Quantify how much the correlation changes
df = pd.DataFrame({
    "Advertising": [10, 15, 20, 25, 30, 35, 40, 45, 50, 55],
    "Sales": [20, 28, 35, 42, 50, 58, 65, 72, 80, 88]
})
corr_before = df.corr()

print("Correlation Before Adding Outlier:")
print(corr_before.round(4))

df_new = pd.concat([
    df,
    pd.DataFrame({"Advertising": [100], "Sales": [10]})
], ignore_index=True)

corr_after = df_new.corr()

print("\nCorrelation After Adding Outlier:")
print(corr_after.round(4))


before = corr_before.loc["Advertising", "Sales"]
after = corr_after.loc["Advertising", "Sales"]

print("\nCorrelation Before:", round(before, 4))
print("Correlation After:", round(after, 4))
print("Change in Correlation:", round(after - before, 4))
print("Absolute Change:", round(abs(after - before), 4))


fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.heatmap(
```

```
    corr_before, annot=True, cmap="coolwarm",
    fmt=".2f", vmin=-1, vmax=1, ax=axes[0]
)
axes[0].set_title("Before Adding Outlier")

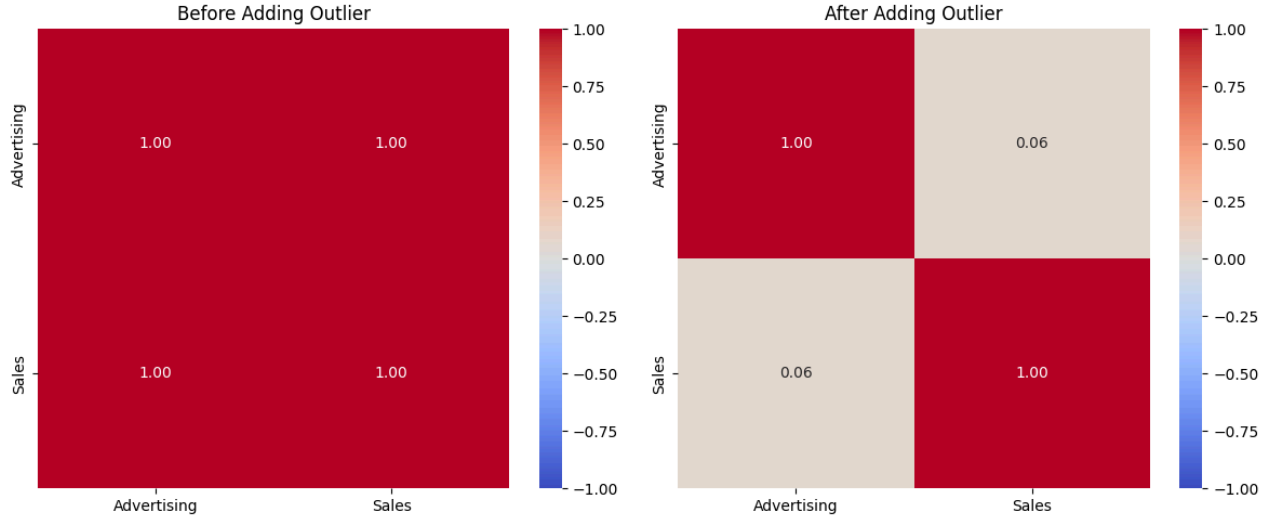
sns.heatmap(
    corr_after, annot=True, cmap="coolwarm",
    fmt=".2f", vmin=-1, vmax=1, ax=axes[1]
)
axes[1].set_title("After Adding Outlier")

plt.tight_layout()
plt.show()
```

```
Correlation Before Adding Outlier:
             Advertising   Sales
Advertising       1.0000  0.9999
Sales             0.9999  1.0000

Correlation After Adding Outlier:
             Advertising   Sales
Advertising       1.0000  0.0647
Sales             0.0647  1.0000

Correlation Before: 0.9999
Correlation After: 0.0647
Change in Correlation: -0.9352
Absolute Change: 0.9352
```




```
# (b) Write Python code to flag potential outliers and compare the correlation before and after adding the unusual observati
#  Explain why an extreme point can distort the apparent relationship and why it should be investigated before being removed


df_new = pd.concat([
    df,
    pd.DataFrame({"Advertising": [100], "Sales": [10]})
], ignore_index=True)
for column in ["Advertising", "Sales"]:
    Q1 = df_new[column].quantile(0.25)
    Q3 = df_new[column].quantile(0.75)
    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    outliers = df_new[
        (df_new[column] < lower_limit) |
        (df_new[column] > upper_limit)
    ]

    print(f"\nPotential outliers in {column}:")
    print(outliers if not outliers.empty else "None")


corr_before = df["Advertising"].corr(df["Sales"])
corr_after = df_new["Advertising"].corr(df_new["Sales"])
```

```
print("\nCorrelation before:", round(corr_before, 4))
print("Correlation after:", round(corr_after, 4))

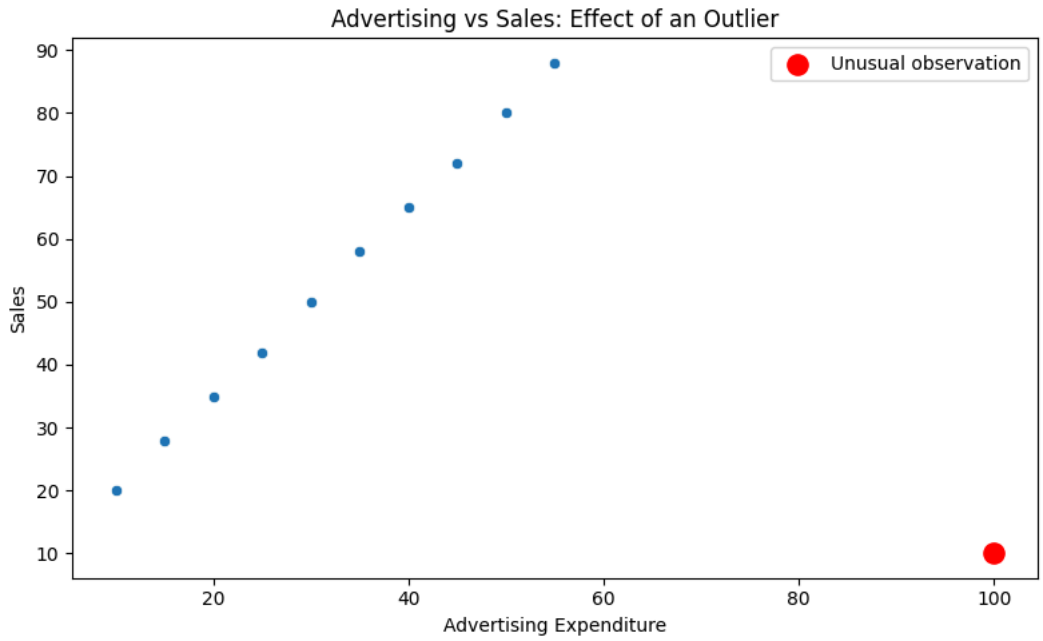
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df_new, x="Advertising", y="Sales")
plt.scatter(100, 10, color="red", s=120, label="Unusual observation")

plt.title("Advertising vs Sales: Effect of an Outlier")
plt.xlabel("Advertising Expenditure")
plt.ylabel("Sales")
plt.legend()
plt.tight_layout()
plt.show()
```

```
Potential outliers in Advertising:
    Advertising  Sales
10          100     10

Potential outliers in Sales:
None

Correlation before: 0.9999
Correlation after: 0.0647
```



Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.